

DevOps Code Review Handbook

**PRACTICAL GUIDANCE FOR REVIEWING CODE,
INFRASTRUCTURE, AND AUTOMATION IN PRODUCTION**



- ✓ SECURITY
- ✓ RELIABILITY
- ✓ SCALABILITY
- ✓ INFRASTRUCTURE
- ✓ CI/CD
- ✓ OPERATIONS
- ✓ BEST PRACTICES



Instagram
@veriqta



GitHub
@veriqta



LinkedIn
@veriqta



Telegram
@veriqta



Medium
@veriqta



X
@veriqta

Code Review Is a Production Control

DevOps code review is not just about finding bugs. It is a **production control** that protects reliability, security, change management and reduces blast radius.

WHY CODE REVIEW EXISTS



Reliability

- Prevents outages
- Identifies failure risks
- Improves system stability



Security

- Finds insecure code
- Prevents credential leaks
- Enforces least privilege



Change Management

- Ensures controlled changes
- Maintains consistency
- Reduces unintended impact



Blast Radius Control

- Identifies what can be affected
- Limits scope of change
- Prevents cascading failures

HOW CODE REVIEW FITS INTO THE DELIVERY FLOW



Developer
Creates Change



Pull Request
Opened



Code Review
(Technical +
Operational)



Approved
and Merged



Deployed
to Production

WHAT SENIOR ENGINEERS LOOK FOR BEYOND SYNTAX

- ✓ Overall design and architecture impact
- ✓ Operational risks and failure modes
- ✓ Security implications
- ✓ Performance and scalability considerations
- ✓ Infrastructure and cloud resource changes
- ✓ Observability and logging
- ✓ Deployment and rollback strategy
- ✓ Maintainability and long-term ownership

REAL WORLD IMPACT

- ⚠ **Prevent** misconfigured infrastructure.
- ⚠ **Catch** over-privileged permissions.
- ⚠ **Stop** changes that can trigger outages.
- ⚠ **Reduce** security vulnerabilities.
- ⚠ **Control** the blast radius of a change.
- ⚠ **Improve** reliability and system resilience.
- ⚠ **Ensure** changes are observable.
- ⚠ **Maintain** consistency across environments.



SENIOR ENGINEER TIP

A good code review thinks about production consequences, not just whether the code works.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Review the Change, Not Just the Diff

A code diff only shows what changed. A good review starts with **intent**. Understand the problem, the bigger context, and the production impact before reading individual lines.

1. START WITH THE INTENT

- What problem is this solving?
- Why is the change needed now?
- Is there an existing incident, ticket or business requirement?
- Does the solution match the problem?

Example: Code Diff (Partial View)

```
10 - replicaCount: 2
11 + replicaCount: 4
12   image: myapp:1.0
13   ports:
14     - - containerPort: 8080
15     + - containerPort: 9090
16
```

 The diff shows these lines, but not the full impact. Always look beyond the diff.

2. UNDERSTAND THE ARCHITECTURE



- Which services or components are affected?
- What environments are involved?
- Does it touch shared infrastructure?
- Are there external dependencies?

3. CHECK KEY DETAILS

	Dependencies	What other systems will be impacted?
	Expected Behavior	What should happen after deployment?
	Deployment Path	How will this be deployed?
	Rollback Strategy	Can we safely roll back? What is the process?

4. IDENTIFY WHAT THE DIFF DOES NOT SHOW

- ! Configuration outside the repository
- ! Dependencies and downstream impact
- ! Environment-specific values
- ! Secrets, permissions and access changes
- ! Database schema or data migration impact
- ! Operational and monitoring changes
- ! Rollout, rollback or manual steps
- ! Infrastructure changes triggered by this code

5. SENIOR ENGINEER QUESTIONS

- ? Does this fully solve the problem?
- ? What is the production impact?
- ? What could go wrong?
- ? How will we detect issues?
- ? Is the rollback path tested and realistic?
- ? Are security and compliance requirements met?
- ? Do we have the right observability in place?
- ? Is this change the simplest and safest approach?



PRODUCTION NOTE

Always review the intent, architecture, and operational impact. The diff is only one piece of the story.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Classify Change Risk Before Reviewing

Not all changes carry the same risk. Classify the risk upfront based on production reach, privilege level, statefulness, reversibility, customer impact, shared infrastructure, data impact and failure-domain expansion. Match the review depth and approval process to the risk level.

Factor	Low Risk (Examples)	Medium Risk (Examples)	High Risk (Examples)
 Production Reach	Dev or non-production	Single production service	Multiple services or global
 Privilege Level	No elevated privileges	Some elevated access	High or wide privileges (e.g. admin, root, IAM)
 Statefulness	Stateless components	Limited state or cache	Stateful systems (database, queue, storage)
 Reversibility	Easy to roll back	Rollback possible with some effort	Hard or unsafe to roll back
 Customer Impact	No customer impact	Minor user impact	Significant customer impact
 Shared Infrastructure	Isolated service	Uses shared services (e.g. database, queue)	Changes core shared infra (e.g. VPC, Kubernetes, IAM)
 Data Impact	No data changes	Read-only or non-critical schema changes	Critical data changes or migrations
 Failure-Domain Expansion	No new dependencies	Expands within same failure domain	Expands across regions, clusters or accounts

EXAMPLE CHANGES AT EACH RISK LEVEL

LOW RISK	MEDIUM RISK	HIGH RISK
- Documentation change - UI text update - Non-critical config - Bug fix in isolated service - Add a new monitoring metric - Update CI comments - Refactor without behavior change	- Deployment config change - New environment variable - Add a new microservice - Update autoscaling settings - Modify ingress rules - Change to shared library - Add a new IAM policy with limited scope	- Database schema migration - Change to VPC, network or security groups - Delete or replace cloud resources - Changes to core Kubernetes components - Modify authentication or authorization - Changes impacting multiple services - Data deletion or irreversible changes

MATCH REVIEW DEPTH TO RISK

Risk Level	Review Requirements
Low Risk	- Standard review - Focus on correctness - Verify basic tests - Single reviewer is often enough
Medium Risk	- Deeper technical review - Check operational impact - Verify monitoring and rollback plan - Involve relevant owners
High	- Multi-engineer review - Architecture and security review - Verify testing, rollback and recovery - Confirm observability and alerting - Require explicit approval from relevant stakeholders



SENIOR ENGINEER TIP

Classifying risk early helps you spend time where it matters. A small diff can still be high risk if the impact is large.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Map the Blast Radius

Before approving a change, determine which services, clusters, accounts, regions, networks, tenants, environments and data paths can be affected. Distinguish **direct impact** from **downstream** and **shared-dependency impact**.

WHAT TO ANALYZE



Services

Which services are changed?



Clusters

Which clusters and namespaces?



Accounts

Which cloud accounts?



Regions

Which regions or AZs?



Networks

Which VPCs, subnets, firewalls?



Tenants

Which customers or tenants?



Environments

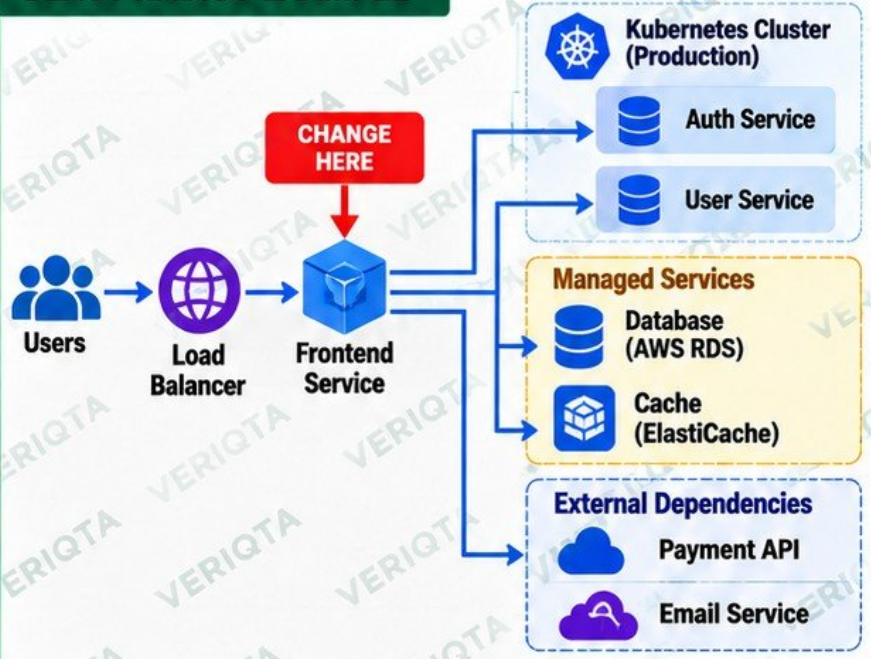
Dev, staging, production?



Data Paths

Which databases, queues, storage?

BLAST RADIUS EXAMPLE



Direct Impact

- Changed service
- Its configuration
- Its deployment



Downstream Impact

- Services it calls
- Databases and queues
- External APIs
- Background jobs



Shared Dependency Impact

- Shared clusters
- Shared VPC or network
- Shared databases
- Shared IAM roles
- Shared infrastructure

KEY QUESTIONS

- ✓ What is the full scope of the change?
- ✓ Which services depend on this?
- ✓ Are there cross-region or cross-account dependencies?
- ✓ Could this change affect other teams or customers?
- ✓ Is there a single point of failure being introduced?
- ✓ What data could be impacted?
- ✓ What happens if this change fails or is rolled back?



SENIOR ENGINEER TIP

Always ask: If this change fails, what else breaks?
Draw the dependency map before approving.

REAL WORLD EXAMPLES

- ⚠ A backend config change affects multiple services.
- ⚠ A Terraform change in a shared VPC impacts all teams.
- ⚠ A Kubernetes change in a shared cluster affects unrelated workloads.
- ⚠ A database schema change breaks multiple applications.
- ⚠ A network rule change blocks critical external APIs.
- ⚠ A small IAM change causes permission errors for other services.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

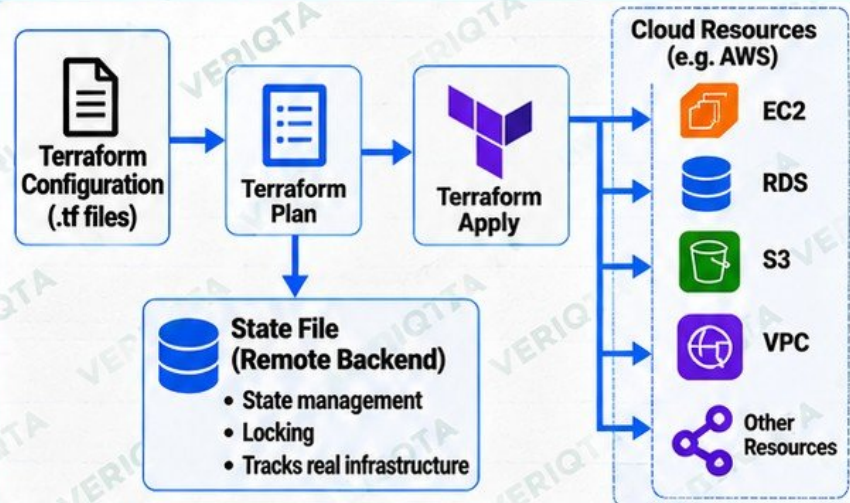
Infrastructure as Code Review

Review **Terraform** and similar **laC** for resource lifecycle, dependencies, state implications, provider behavior, drift, naming, tagging, module boundaries, environment separation, and unintended infrastructure changes.

KEY AREAS TO REVIEW

-  **Resource Lifecycle**
Create, update, destroy, replace.
-  **Dependencies**
Use explicit dependencies where needed.
-  **State Implications**
Check for state changes, locking and remote backend.
-  **Provider Behavior**
Understand how the provider handles updates and replacements.
-  **Drift**
Detect and prevent configuration drift.
-  **Naming and Tagging**
Follow consistent naming and tagging standards.
-  **Module Boundaries**
Ensure modules are reusable, focused and well-defined.
-  **Environment Separation**
Keep dev, staging and production isolated.
-  **Unintended Changes**
Look for resources that could be accidentally modified or destroyed.

TERRAFORM ARCHITECTURE (SIMPLIFIED)



EXAMPLE: REVIEWING A RESOURCE CHANGE

```
resource "aws_instance" "web" {
  ami           = "ami-0abc1234"
  instance_type = "t3.large" # changed
  tags = {
    Name     = "web-prod"
    Environment = "prod"
    Owner    = "platform"
  }
}
```

WHAT TO CHECK

- Is the change expected?
- Will this replace the resource?
- What is the impact on state?
- Are tags correct?
- Is the instance type allowed?
- Does this affect cost or capacity?
- Are there dependencies?
- Could this impact other services?

COMMON RISKY CHANGES

- ❗ Accidental resource destruction
- ❗ Unintended replacement (ForceNew)
- ❗ Changing critical network settings
- ❗ Modifying shared infrastructure
- ❗ Removing important tags
- ❗ Using incorrect provider or region
- ❗ Hard-coded values instead of variables
- ❗ Changes that bypass environment separation

BEST PRACTICES

- ✓ Review the Terraform plan output
- ✓ Use remote state with locking
- ✓ Use consistent naming and tagging
- ✓ Separate environments and accounts
- ✓ Use approved modules
- ✓ Validate input variables
- ✓ Check for least privilege IAM roles
- ✓ Consider drift detection
- ✓ Document the change intent



SENIOR ENGINEER TIP

The code diff can look small, but the Terraform plan may show a large blast radius.

Always review the plan to understand the real infrastructure impact before approval.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Terraform Plan Under the Microscope

The **Terraform plan** shows what will actually happen. Read the plan as evidence and look for create, update, destroy and replacement operations, ForceNew behavior, unknown values, sensitive changes and dependency effects.

1. UNDERSTAND PLAN ACTIONS

-  **Create (+)**
New resource will be created.
-  **Update (~)**
Existing resource will be modified in place.
-  **Destroy (-)**
Resource will be deleted.
-  **Replace (+/-)**
Resource will be destroyed and recreated (usually due to ForceNew).

3. KEY THINGS TO CHECK

-  **ForceNew Behavior**
Some changes require resource replacement (e.g. instance_type, subnet_id, engine_version).
-  **Unknown Values**
(known after apply) can hide important changes.
-  **Sensitive Changes**
Look for credentials, keys or other sensitive values.
-  **Dependencies**
One change can trigger updates across many resources.
-  **State Implications**
Check if state will be moved, locked or modified.
-  **Provider Behavior**
Different providers may handle updates and replacements differently.

6. BEST PRACTICES

- ✓ Always review the full plan output.
- ✓ Look for replacements, not just in-place updates.
- ✓ Validate unknown values.
- ✓ Check dependencies and downstream impact.
- ✓ Review changes in a non-production environment first.
- ✓ Confirm the plan matches the intended change.
- ✓ Use tagging and naming standards.



PRODUCTION NOTE

Never approve a change based only on the code diff. Always review and understand the full Terraform plan. A clean diff can still have a large blast radius.

2. EXAMPLE: TERRAFORM PLAN OUTPUT

```
# aws_instance.web will be created
+ resource "aws_instance" "web" {
+   ami           = "ami-0abc1234"
+   instance_type = "t3.large"
+   tags          = {
+     Name = "web-prod"
+   }
}

# aws_security_group.app will be updated in place
~ resource "aws_security_group" "app" {
~   description = "Allow app traffic"
~   ingress    = [ ... ]
}

# aws_db_instance.db must be replaced
-/+ resource "aws_db_instance" "db" {
~   instance_class = "db.t3.medium" -> "db.t3.large"
}
```

Create
New resource

Update
In place

Replace
Destroy and create

4. WHY A SMALL DIFF CAN BE DANGERIOUS

- A small code change can trigger resource replacement.
- Changing a default value may update multiple resources.
- Provider schema changes can modify behavior.
- Dependencies can cause cascading updates.
- Unknown values can hide large infrastructure changes.
- Tags, network settings or IAM changes can impact security and connectivity.

5. EXAMPLE: HIDDEN IMPACT FROM A SMALL CHANGE

Code Diff (looks small)

```
- instance_type = "t3.medium"
+ instance_type = "t3.large"
```

Terraform Plan (big impact)

- /+ Resource will be replaced
- Instance will be destroyed
- + New instance will be created
- New private IP address
- Possible service interruption

7. SENIOR ENGINEER QUESTIONS

- ? What resources will be created, updated, destroyed or replaced?
- ? Is any replacement expected or avoidable?
- ? Could this impact other environments or teams?
- ? Are there unknown values that need clarification?
- ? Does this change affect cost, security or availability?
- ? Have dependent resources been reviewed?
- ? Is the state update safe and reversible?



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Kubernetes Manifest Review

Review Deployments, StatefulSets, Services, ConfigMaps, Secrets references, probes, resources, security contexts, affinity, topology constraints, PDBs, and rollout configuration from a production perspective.

1. KEY RESOURCES TO REVIEW



Deployments

Replica count, strategy, containers, env vars, probes, resources, security context.



StatefulSets

Stable identity, storage, ordering, update strategy.



Services

Type, ports, selectors, external access, session handling.



ConfigMaps

Non-sensitive configuration, change management.



Secrets

Secrets references, not hard-coded, correct access.



Probes

Liveness, readiness, startup probes, thresholds.



Resources

CPU and memory requests and limits, preventing OOM and throttling.



Security Contexts

Run as non-root, capabilities, file system access.



Affinity & Topology

Node affinity, pod anti-affinity, topology spread constraints.



Pod Disruption Budgets

Maintain availability during disruptions.



Rollout Configuration

Rolling updates, maxUnavailable, maxSurge, rollback strategy.

2. EXAMPLE: DEPLOYMENT MANIFEST

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: production
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginx:1.25
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          livenessProbe:
            httpGet:
              path: /healthz
              port: 80
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /ready
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10
```

3. COMMON REVIEW CHECKLIST

- ✓ Are replicas and rollout strategy correct?
- ✓ Are resource requests and limits set?
- ✓ Are liveness, readiness and startup probes configured?
- ✓ Are ConfigMaps and Secrets used correctly?
- ✓ Is the container running as non-root?
- ✓ Are security contexts and capabilities restricted?
- ✓ Are affinity and topology constraints appropriate?
- ✓ Is a PodDisruptionBudget defined?
- ✓ Are environment variables and configuration managed safely?
- ✓ Are images from trusted registries and properly versioned?
- ✓ Are annotations, labels and naming consistent?
- ✓ Will this change maintain availability during node or cluster upgrades?

4. COMMON ISSUES TO LOOK FOR

- ! Missing or incorrect probes
- ! No resource limits (risk of OOM)
- ! Running as root or privileged containers
- ! Hard-coded secrets in manifests
- ! Using latest tags instead of specific versions
- ! No PodDisruptionBudget for critical services
- ! Unintended public exposure via Service type
- ! Incorrect selectors leading to traffic issues
- ! Configuration changes that cause restarts
- ! Missing persistence settings for stateful apps
- ! Affinity rules that reduce availability

5. ROLLOUT STRATEGIES



Rolling Update

Default strategy, gradual replacement.



Recreate

All pods replaced at once (downtime).



Blue-Green

Switch between environments.



Canary

Roll out to a small percentage first.



SENIOR ENGINEER TIP

Always review Kubernetes manifests as if the change is going to a live production cluster. A small misconfiguration can cause an outage, data loss or security exposure.

6. PRODUCTION REVIEW QUESTIONS

- ? Will this change cause unintended restarts?
- ? Is availability maintained during rollout?
- ? Are dependencies (ConfigMaps, Secrets) in place?
- ? Does it meet security and compliance requirements?
- ? How will this behave during a node failure?
- ? Is there a rollback plan if the deployment fails?



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Resource Limits, Probes and Hidden Failure Modes

How bad CPU and memory settings create throttling, OOMKills, and scheduling problems. Readiness versus liveness probes. How innocent configuration can create cascading restarts or remove healthy capacity.

1. CPU AND MEMORY SETTINGS

What They Do



CPU

- Controls how much CPU the container can use.
- Prevent noisy neighbours.
- Avoid CPU throttling.



Memory

- Sets minimum (request) and maximum (limit).
- Prevents OOMKills.
- Affects scheduling on nodes.

RESOURCE CONFIGURATION EXAMPLE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  template:
    spec:
      containers:
        - name: app
          image: nginx:1.25
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

Minimum guaranteed resources for scheduling

Maximum resources the container can use

COMMON PROBLEMS



CPU Throttling

Causes slow response times and increased latency.



OOMKills

Container is killed when memory limit is exceeded.



Scheduling Failures

Pods stay in Pending due to insufficient resources.



Cascading Restarts

Unhealthy probe or low resources can trigger continuous restarts.



Reduced Capacity

Aggressive probes or high requests can remove healthy pods from service.

2. PROBE TYPES



Liveness Probe

- Checks if the container is alive.
- If it fails, Kubernetes restarts the container.
- Use for real deadlock or hang scenarios.



Readiness Probe

- Checks if the container is ready to serve traffic.
- If it fails, pod is removed from Service endpoints.
- Use for dependencies (DB, API).



Startup Probe

- Gives the container time to start before liveness/readiness checks.
- Use for slow starting applications.
- Prevents premature restarts.

PROBE CONFIGURATION EXAMPLE

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
  failureThreshold: 3
  restart if unhealthy

readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 3
  remove from service if not ready

startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
  allow more time to start
```

3. HIDDEN FAILURE MODES

- Low CPU limit causes throttling → slow responses and timeouts.
- Low memory limit causes OOMKills → container restarts.
- Probes configured too aggressively → healthy pods marked unhealthy.
- Startup time longer than probe delay → continuous restarts.
- Readiness probe depends on an external service → removes healthy pods during dependency outage.
- High resource requests → pods cannot be scheduled → reduced capacity.
- Misconfigured probes → cascading restarts across multiple pods.

4. BEST PRACTICES

- Set realistic CPU and memory requests and limits.
- Use all three probes where appropriate.
- Tune probe delays and thresholds for your application.
- Monitor CPU throttling, memory usage, and restart counts.
- Test probe behavior during dependency failures.
- Review resource configuration for each environment.

5. SIGNS OF MISCONFIGURATION

- Frequent container restarts.
- Pods flapping between ready and not ready.
- High CPU throttling metrics.
- OOMKilled events in pod logs.
- Pods stuck in Pending state.
- Services losing endpoints unexpectedly.



SENIOR ENGINEER TIP

Resource limits and probes are not just configuration. They directly impact availability, stability and capacity. A small change can remove healthy pods or trigger a cascade during peak traffic.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CI/CD Pipeline Code Review

Review workflow triggers, stages, dependencies, runners, permissions, artifact handling, environment promotion, concurrency, approvals, failure handling, retries, and rollback behavior. Treat the pipeline as production infrastructure.

1. KEY AREAS TO REVIEW



Workflow Triggers

When the pipeline runs and what events start it.



Stages and Dependencies

Build, test, security, deploy, and stage order.



Runners

Runner type, trust level, isolation and required tools.



Permissions

Secrets, cloud credentials, and least privilege access.



Artifact Handling

What is built, stored, retained, and promoted.



Environment Promotion

Dev → Staging → Production and required approvals.



Concurrency

Parallel runs, cancellations and environment locks.



Approvals

Manual approvals and policy checks before deployment.



Failure Handling & Retries

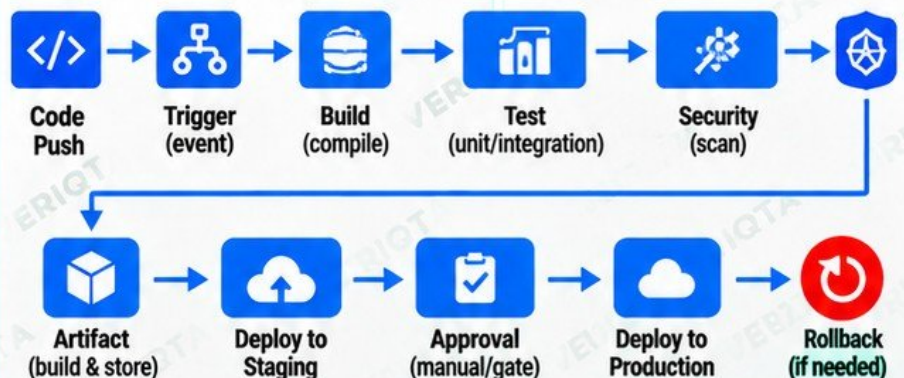
How failures are detected, retried and alerted.



Rollback Behavior

How to quickly and safely revert changes if something fails.

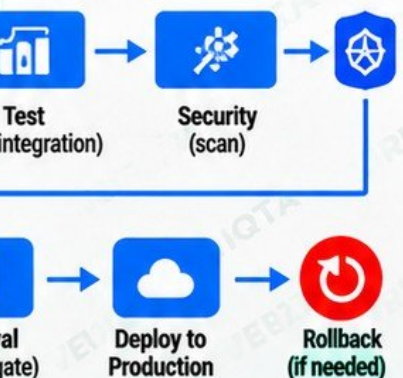
2. EXAMPLE: CI/CD PIPELINE FLOW



3. SAMPLE PIPELINE CONFIGURATION (GitHub Actions)

```
name: CI-CD
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup
        uses: actions/setup-python@v5
      - name: Build
        run: |
          echo "Building application..."
  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: pytest
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    environment: production
    steps:
      - name: Deploy to production
        run: ./deploy.sh
```

4. COMMON ISSUES TO LOOK FOR



5. BEST PRACTICES

- ✓ Use explicit and minimal permissions.
- ✓ Protect main and release branches.
- ✓ Include security scanning (SAST/DAST, dependency, container).
- ✓ Require approvals for production.
- ✓ Version and sign artifacts.
- ✓ Use environment separation and locks.
- ✓ Set timeouts and retry limits.
- ✓ Test rollback regularly.
- ✓ Monitor pipeline health and alert on failures.
- ✓ Document the pipeline and changes.



SENIOR ENGINEER TIP

A CI/CD pipeline is production infrastructure. Code review should treat pipeline changes with the same attention as application and infrastructure changes.



REAL WORLD IMPACT

- A misconfigured pipeline can deploy untested code to production.
- Leaked credentials can be exploited.
- A failed rollout without rollback can cause a major outage.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Secrets, Identity and Privilege Review

Detect embedded credentials, overprivileged service accounts, unsafe environment variables, long-lived tokens, broad IAM/RBAC permissions, secret leakage through logs, and dangerous trust relationships. Prefer workload identity and least privilege.

1. KEY AREAS TO REVIEW



Embedded Credentials

Look for hard-coded keys, passwords, tokens and certificates in code, configs and scripts.



Service Accounts

Check for overprivileged service accounts (cloud, Kubernetes, CI/CD, applications).



Environment Variables

Identify sensitive values in env vars, docker files and configuration files.



Token Management

Detect long-lived tokens and static credentials. Prefer short-lived tokens.



IAM / RBAC Permissions

Review for least privilege and remove unnecessary permissions.



Secret Leakage

Check for secrets in logs, error messages, monitoring and artifacts.



Trust Relationships

Review cross-account, cross-cluster and external trust relationships.



Preferred Approach

Use workload identity, short-lived credentials and least privilege.

2. EXAMPLES OF RISKY CONFIGURATIONS

Hard-coded credentials

```
aws_access_key_id =  
  "AKIAIOSFODNN7EXAMPLE"  
aws_secret_access_key =  
  "wJalrXUtnFEMI/K7MDENG/bPxrFi  
  CYEXAMPLEKEY"
```

Overprivileged IAM policy

```
{  
  "Effect": "Allow",  
  "Action": "*",  
  "Resource": "*" }  
}
```

Secrets in environment variables

```
env:  
  - name: DB_PASSWORD  
    value: "SuperSecret123!"
```

Secrets in logs

```
INFO Connecting to database  
INFO Password=SuperSecret123!  
INFO Connection successful
```

3. SECURE ALTERNATIVES

Workload Identity (instead of static keys)



- ✓ No hard-coded credentials
- ✓ Short-lived tokens
- ✓ Least privilege access
- ✓ Automatic rotation

Use Secret Management



- ✓ Store secrets in a vault (AWS Secrets Manager, HashiCorp Vault, etc.)
- ✓ Use runtime injection
- ✓ Rotate secrets regularly
- ✓ Never log secret values

Least Privilege IAM / RBAC



- ✓ Grant only required permissions
- ✓ Use scoped roles and policies
- ✓ Separate environments (dev/staging/prod)
- ✓ Review and remove unused permissions
- ✓ Use Kubernetes RBAC and service accounts correctly

4. COMMON ISSUES TO LOOK FOR

- ❗ Hard-coded API keys or passwords
- ❗ Overprivileged service accounts
- ❗ Secrets in environment variables
- ❗ Long-lived tokens or static credentials
- ❗ Broad IAM/RBAC permissions
- ❗ Secrets exposed in logs or error messages
- ❗ Artifacts containing sensitive data
- ❗ Dangerous cross-account or external trust
- ❗ Missing rotation for keys and certificates
- ❗ Using default or shared credentials

5. REVIEW CHECKLIST

- ✓ Are there any hard-coded credentials?
- ✓ Are secrets stored securely?
- ✓ Are environment variables safe?
- ✓ Are tokens short-lived?
- ✓ Are IAM/RBAC permissions minimized?
- ✓ Could secrets leak through logs or artifacts?
- ✓ Are service accounts correctly scoped?
- ✓ Are trust relationships necessary and safe?
- ✓ Is workload identity used where possible?
- ✓ Are rotation and expiration policies in place?



SENIOR ENGINEER TIP

If a compromise of this code gives an attacker excessive access, the problem is not just in the code.

Review secrets, identity and privileges as if you're reviewing the production infrastructure itself.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Supply Chain and Dependency Risk

Review base images, package dependencies, GitHub Actions or pipeline dependencies, image provenance, version pinning, registries, build inputs, SBOMs, signing, scanning, and dependency trust boundaries.

1. KEY AREAS TO REVIEW



Base Images

Use trusted, minimal and up-to-date base images. Avoid unverified or unmaintained images.



Package Dependencies

Check versions, known vulnerabilities and maintainers.



Pipeline Dependencies

Review GitHub Actions, plugins and third-party actions.



Version Pinning

Use specific versions or digests. Avoid latest tags.



Registries

Use trusted registries. Validate image sources and access controls.



Build Inputs

Review external downloads, URLs, scripts and artifacts.



SBOMs

Generate and review SBOMs (Software Bill of Materials).



Signing and Verification

Verify image and artifact signatures.



Trust Boundaries

Understand what you trust and what comes from outside your organization.

2. EXAMPLE: DOCKERFILE REVIEW



```
# Bad: using latest and unverified image
FROM node:latest

# Good: specific version with digest
FROM node:20.11.1@sha256:abc123...

# Install dependencies
COPY package.json package-lock.json ./
RUN npm ci --only=production

# Run as non-root
USER node
```

Avoid latest tag
(unpredictable changes)

Use specific version
and digest

Run as non-root
for better security

3. DEPENDENCY FLOW



4. COMMON RISKS

- ⚠ Using latest tags for base images or packages
- ⚠ Dependencies with known vulnerabilities
- ⚠ Untrusted or unverified GitHub Actions
- ⚠ Compromised or malicious packages
- ⚠ No version pinning or digest validation
- ⚠ Pulling images from public or unknown registries
- ⚠ Missing SBOMs and vulnerability scanning
- ⚠ Unsigned or unverified artifacts and images
- ⚠ Hidden build inputs (scripts, remote downloads)
- ⚠ Dependency confusion (internal vs public)
- ⚠ Overly permissive access to registries
- ⚠ Trusting third-party code without review

5. BEST PRACTICES

- ✓ Use minimal, official and maintained base images
- ✓ Pin versions and use digests
- ✓ Scan images and dependencies regularly
- ✓ Generate and review SBOMs
- ✓ Verify signatures (images and artifacts)
- ✓ Use trusted registries and private repositories
- ✓ Review GitHub Actions and third-party code
- ✓ Limit and monitor build inputs
- ✓ Separate trusted and untrusted sources
- ✓ Implement least privilege for registry access
- ✓ Automate dependency updates and alerts
- ✓ Have a process for dependency approval



SENIOR ENGINEER TIP

Your supply chain is part of your production environment. A compromised dependency can give an attacker access to your systems. Treat dependencies, images and build pipelines with the same scrutiny as your application code.



REAL WORLD EXAMPLE

A single compromised GitHub Action or a vulnerable base image has caused major breaches, credential theft and widespread supply chain attacks across many organizations.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Networking Changes Need Different Scrutiny

Review DNS, firewall/security-group rules, routes, ingress, egress, load balancers, NetworkPolicy, TLS, proxies, ports, NAT, and service discovery. Look for connectivity failures and accidental exposure.

1. KEY AREAS TO REVIEW



DNS

Check records, TTL, and correct targets.



Firewall / Security Groups

Review inbound and outbound rules. Look for overly broad access.



Routes

Check routes, route tables and peering.



Ingress / Load Balancers

Review listener rules, target groups, health checks and certificates.



NetworkPolicy

Validate pod-to-pod and namespace rules.



TLS / Certificates

Check certificate configuration, expiry and allowed protocols.



Proxies

Review proxy rules, bypass lists and authentication.



Ports

Ensure only required ports are open.



NAT

Check NAT gateways, egress IPs and IP allow lists.



Service Discovery

Validate internal and external service discovery configurations.



Egress

Review outbound access and external dependencies.

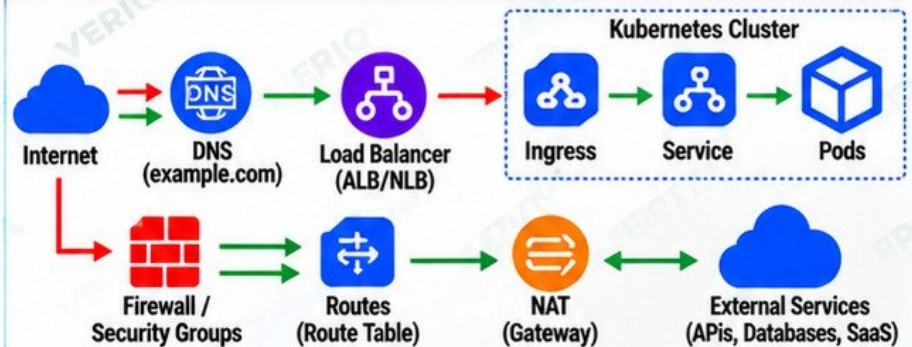
2. EXAMPLE: SECURITY GROUP CHANGE (AWS)

```
resource "aws_security_group" "app" {
  name = "app-sg"
  ingress {
    from_port = 80
    to_port   = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # RISKY
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Allows HTTP from anywhere (0.0.0.0/0) → potential accidental exposure

Restrict to specific CIDR ranges or use a load balancer instead

3. NETWORKING CHANGE IMPACT DIAGRAM



4. COMMON ISSUES TO LOOK FOR

- Overly permissive firewall/security-group rules
- Accidental exposure of internal services
- Incorrect DNS records or low TTL changes
- Misconfigured ingress or load balancer rules
- Broken routes or missing network peering
- Incorrect NetworkPolicy rules
- Expired or incorrect TLS certificates
- Unintended egress to the internet
- Wrong NAT or IP allow list configuration
- Missing or incorrect service discovery entries
- Proxy misconfiguration causing connectivity failures

5. REVIEW CHECKLIST

- Are firewall rules least privilege?
- Are DNS records correct and intentional?
- Are routes and peering configurations valid?
- Are ingress and load balancer rules secure?
- Are NetworkPolicy rules correct?
- Are TLS certificates valid and up to date?
- Are only required ports open?
- Is egress access controlled?
- Are NAT and IP allow lists configured correctly?
- Are service discovery entries accurate?
- Have you tested connectivity before merge?
- Could this change expose internal services?

SENIOR ENGINEER TIP

Networking changes can silently break production or expose internal services to the internet.

Always review networking changes with the same level of scrutiny as application code.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Reliability Review

Ask what happens when dependencies fail. Review timeouts, retries, exponential backoff, jitter, circuit breaking, health checks, redundancy, graceful degradation, rate limiting, queues, and failover. Explain how resilience mechanisms can amplify incidents.

1. KEY AREAS TO REVIEW



Timeouts

Ensure sensible timeouts for all external calls.



Retries

Check retry logic, limits and which errors are retried.



Exponential Backoff

Use exponential backoff to avoid overload.



Jitter

Add jitter to prevent thundering herd.



Circuit Breaking

Stop calls when dependencies are unhealthy.



Health Checks

Validate liveness, readiness and dependency checks.



Redundancy

Check multi-AZ/region and replica configuration.



Graceful Degradation

Ensure partial functionality when dependencies fail.



Rate Limiting

Protect services from excessive traffic.



Queues

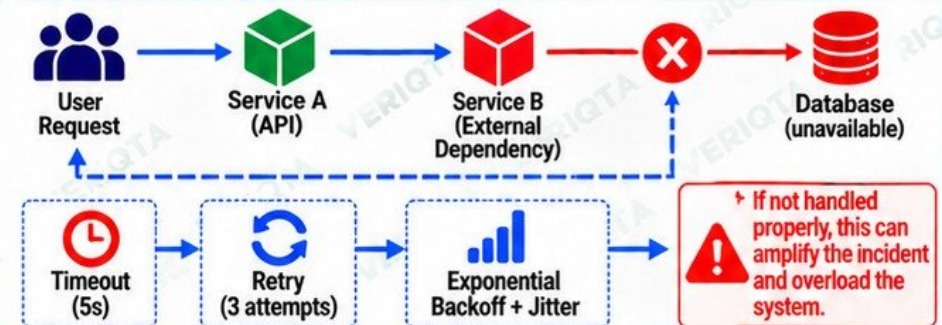
Use queues for buffering and asynchronous processing.



Failover

Verify automatic failover and recovery behavior.

2. DEPENDENCY FAILURE SCENARIO



3. EXAMPLE: RETRY WITH BACKOFF

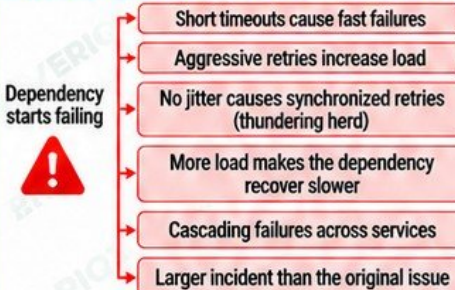
```
import time
import random

def call_service():
    max_retries = 3
    base_delay = 1
    for attempt in range(max_retries):
        try:
            return make_request()
        except Exception as e:
            if attempt == max_retries - 1:
                raise e
            # Exponential backoff with jitter
            delay = (2 ** attempt) + \
                random.uniform(0, 1)
            time.sleep(delay)
```

4. CIRCUIT BREAKER BEHAVIOR



5. HOW RESILIENCE CAN AMPLIFY INCIDENTS



6. BEST PRACTICES

- ✓ Set sensible timeouts for all external calls.
- ✓ Use limited retries with exponential backoff.
- ✓ Add jitter to avoid synchronized retries.
- ✓ Implement circuit breakers.
- ✓ Configure proper health checks.
- ✓ Design for redundancy (multi-AZ/region).
- ✓ Support graceful degradation.
- ✓ Use rate limiting and bulkheads.
- ✓ Use queues for asynchronous processing.
- ✓ Test failover and recovery regularly.

7. COMMON ISSUES TO LOOK FOR

- | | |
|-------------------------------------|--------------------------------------|
| ❗ No timeouts or very long timeouts | ❗ Single point of failure |
| ❗ Unlimited or aggressive retries | ❗ No graceful degradation |
| ❗ No exponential backoff or jitter | ❗ No rate limiting |
| ❗ Missing circuit breaker | ❗ Queues not used or misconfigured |
| ❗ Dependent services not checked | ❗ Failover not tested or not working |



SENIOR ENGINEER TIP

Resilience mechanisms are essential, but when misconfigured, they can amplify incidents by increasing load, consuming resources, and cascading failures. Always test failure scenarios and observe how the system behaves under stress.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

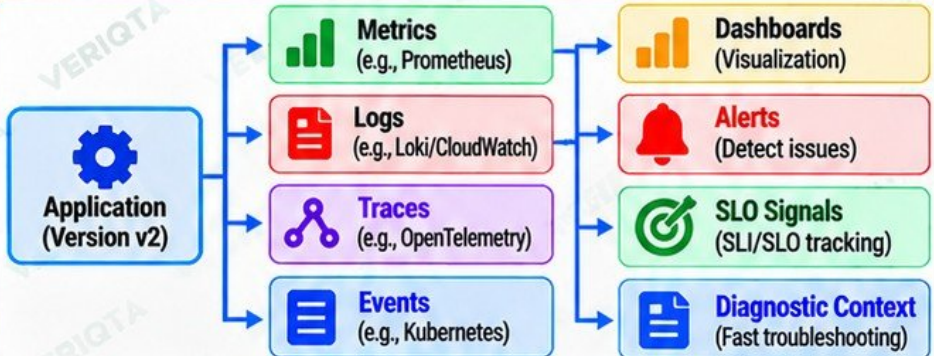
Observability Before Approval

Determine whether the change will be observable after deployment. Metrics, structured logs, traces, events, dashboards, alerts, deployment markers, SLO signals, and diagnostic context. Ask, "How will we know this change is failing?"

1. KEY AREAS TO REVIEW

-  **Metrics**
Check what metrics will be available after deployment (e.g., RED/USE metrics).
-  **Structured Logs**
Ensure logs are structured, with useful fields and no sensitive data.
-  **Traces**
Verify distributed tracing is enabled and carries context (e.g., trace IDs).
-  **Events**
Check application and infrastructure events (e.g., Kubernetes events).
-  **Dashboards**
Confirm dashboards exist or will be updated for the change.
-  **Alerts**
Ensure alerting is in place for failures and abnormal behavior.
-  **Deployment Markers**
Verify version, commit SHA, and deployment annotations.
-  **SLO Signals**
Confirm SLI/SLO metrics will show impact (latency, errors, availability).
-  **Diagnostic Context**
Check that logs, traces and metrics include enough context to troubleshoot.

2. OBSERVABILITY FLOW AFTER DEPLOYMENT



3. EXAMPLE: STRUCTURED LOG

```
{
  "timestamp": "2026-10-05T14:32:10Z",
  "level": "info",
  "service": "payment-service",
  "version": "v2.1.0",
  "commit": "a1b2c3d",
  "trace_id": "4f9c1d2e3a",
  "span_id": "7e8d9c0b1a",
  "user_id": "12345",
  "message": "Payment processed",
  "duration_ms": 245
}
```

4. EXAMPLE: METRICS (Prometheus)

```
# Request rate
http_requests_total{service="api",
version="v2"} 1024

# Error rate
http_requests_errors_total{service="api",
version="v2"} 12

# Latency
http_request_duration_seconds_bucket{
service="api", version="v2", le="0.5"}
900
```

5. DASHBOARD EXAMPLE

Service Overview (v2.1.0)



6. DEPLOYMENT MARKERS

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
  labels:
    app: payment-service
    version: "v2.1.0"
    commit: "a1b2c3d"
  deployed-at: "2026-10-05T14:30:00Z"
  managed-by: "github-actions"
```

7. COMMON ISSUES TO LOOK FOR

- ❗ No metrics for the new service or version
- ❗ Missing structured logs or useful fields
- ❗ No tracing or missing correlation IDs
- ❗ Dashboards not updated for the change
- ❗ No alerts or incorrect alert thresholds
- ❗ Deployment markers missing (version/commit)
- ❗ SLOs not tracking the new service
- ❗ Insufficient diagnostic context
- ❗ Logs contain sensitive data
- ❗ Events not monitored
- ❗ Alerts are too noisy or too late
- ❗ No visibility into external dependencies

8. REVIEW CHECKLIST

- ✓ Will metrics be available after deployment?
- ✓ Are logs structured and useful?
- ✓ Is distributed tracing enabled?
- ✓ Are dashboards updated?
- ✓ Are alerts configured and tested?
- ✓ Are deployment markers included?
- ✓ Will SLO/SLI signals show impact?
- ✓ Is diagnostic context sufficient?
- ✓ "How will we know this change is failing?"

 **SENIOR ENGINEER TIP**
If you cannot see the impact of a change, you will react late.

Observability is not optional – it is part of a safe deployment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Deployment and Rollback Review

Rolling, blue-green, canary, feature flags, progressive delivery, database compatibility, migration ordering, backward compatibility, rollback feasibility, and irreversible operations. A rollback statement is not the same as a tested rollback path.

1. KEY AREAS TO REVIEW



Deployment Strategy

Understand rolling, blue-green, canary and progressive delivery.



Feature Flags

Verify feature flags are used for safe rollouts and quick disable.



Database Compatibility

Check schema changes, migration ordering and backward compatibility.



Migration Ordering

Verify safe deployment and rollback order for all components.



Backward Compatibility

Ensure new version works with older versions (and vice versa).



Progressive Delivery

Validate staged rollout, traffic shifting and monitoring.



Rollback Feasibility

Confirm a tested and working rollback path exists.



Irreversible Operations

Identify changes that cannot be easily rolled back (e.g., data deletions, schema drops).

2. DEPLOYMENT STRATEGIES

Rolling Deployment



- ✓ Update instances gradually
- ✓ No downtime
- ✓ Monitor during rollout

Blue-Green Deployment



- ✓ Deploy new version in parallel
- ✓ Switch traffic
- ✓ Quick rollback

Canary Deployment



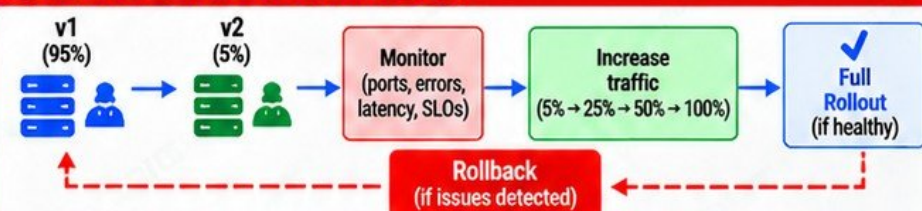
- ✓ Release to small percentage of users
- ✓ Monitor metrics
- ✓ Increase traffic gradually

Feature Flags

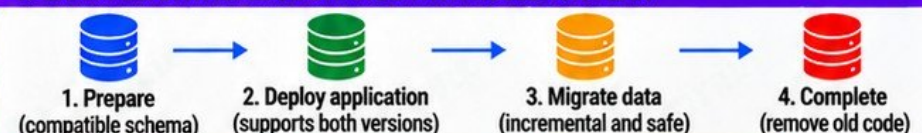


- ✓ Enable/disable features at runtime
- ✓ Test in production
- ✓ Quickly turn off problematic features

3. EXAMPLE: CANARY DEPLOYMENT FLOW



4. EXAMPLE: DATABASE CHANGE WITH ROLLBACK PLAN



Rollback Considerations

- ✓ Use backward compatible schema changes
- ✓ Avoid breaking changes until all instances are updated
- ✓ Test rollback procedure in non-production
- ✓ Have a plan for data migration rollback (if possible)

Dangerous Changes

- ⚠ Dropping columns or tables
- ⚠ Irreversible data transformations
- ⚠ Incompatible schema changes
- ⚠ Deleting critical data
- ⚠ Changing primary keys

5. COMMON ISSUES TO LOOK FOR

- ⚠ No clear deployment strategy
- ⚠ Rollbacks not tested
- ⚠ Database changes without backward compatibility
- ⚠ Incorrect migration order
- ⚠ Feature flags not used for risky changes
- ⚠ Irreversible operations without proper review
- ⚠ No monitoring during rollout
- ⚠ Assuming rollback is just a "git revert" or "helm rollback"
- ⚠ Dependencies not updated in the correct order

6. REVIEW CHECKLIST

- ✓ Is the deployment strategy appropriate?
- ✓ Are feature flags available for risky changes?
- ✓ Are database changes backward compatible?
- ✓ Is migration order safe and documented?
- ✓ Has the rollback path been tested?
- ✓ Are there any irreversible operations?
- ✓ Is monitoring and alerting in place?
- ✓ Are dependencies updated in the right order?
- ✓ Is progressive delivery used where possible?
- ✓ Have success and rollback criteria been defined?



SENIOR ENGINEER TIP

A rollback statement in a pull request does not mean you have a tested rollback path.

Always test the deployment and rollback process in a non-production environment that matches production as closely as possible.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Reviewing Automation and Scripts

Review Bash, Python, Ansible, operational scripts, and automation for idempotency, input validation, error handling, retries, concurrency, destructive operations, privilege, exit codes, partial execution, and safe reruns.

1. KEY AREAS TO REVIEW



Idempotency

Ensure the script can be run multiple times without unintended changes.



Input Validation

Validate and sanitize all inputs to prevent errors and injection risks.



Error Handling

Handle failures properly and provide clear error messages.



Retries

Use retries with backoff for transient failures.



Concurrency

Check for race conditions, locking and parallel execution issues.



Destructive Operations

Identify commands that delete, modify or replace resources.



Privilege and Access

Verify least privilege and avoid unnecessary root access.



Exit Codes

Use correct and consistent exit codes for success and failure.



Partial Execution

Check what happens if the script fails halfway.



Safe Reruns

Ensure the script can be safely re-run after a failure or partial execution.

2. EXAMPLE: BASH SCRIPT REVIEW

```
#!/usr/bin/env bash
set -euo pipefail

# Validate input
if [[ -z "${ENV:-}" ]]; then
    echo "Error: ENV is required (dev|staging|prod)"
    exit 1
fi

# Idempotent example: create directory if not exists
if [[ ! -d "/opt/app" ]]; then
    mkdir -p /opt/app
fi

# Retry with backoff
for i in {1..3}; do
    if deploy_app; then
        echo "Deployment successful"
        break
    else
        echo "Attempt $i failed, retrying in $((2**i)) seconds..."
        sleep $((2**i))
    fi
done
```

Fail on errors, undefined variables and pipe failures

Validate and sanitize inputs

Idempotent operation (checks if exists)

Retry with exponential backoff

3. EXAMPLE: ANSIBLE TASK REVIEW

```
- name: Ensure app is installed
  ansible.builtin.package:
    name: myapp
    state: present
    register: result
    retries: 3
    delay: 10
    until: result is succeeded
```

CHECK FOR

- ✓ Idempotent modules (use state)
- ✓ Proper error handling (retries/until)
- ✓ Input validation (vars)
- ✓ Least privilege (become only if needed)
- ✓ Clear and meaningful task names
- ✓ Avoid hardcoded secrets
- ✓ Safe reruns (no unintended changes)

4. COMMON ISSUES TO LOOK FOR

- ❗ Non-idempotent scripts (different results on rerun)
- ❗ Missing input validation
- ❗ No error handling or retries
- ❗ Unchecked destructive operations (rm, delete, drop)
- ❗ Running with excessive privileges
- ❗ Incorrect or missing exit codes
- ❗ Scripts that fail halfway and leave inconsistent state
- ❗ Race conditions in concurrent execution
- ❗ Hardcoded secrets or sensitive data
- ❗ No logging or insufficient logging

5. BEST PRACTICES

- ✓ Design scripts to be idempotent
- ✓ Validate and sanitize all inputs
- ✓ Use error handling and retries
- ✓ Implement exponential backoff
- ✓ Avoid destructive operations or add confirmation checks
- ✓ Use least privilege
- ✓ Return meaningful exit codes
- ✓ Handle partial execution safely
- ✓ Support safe reruns
- ✓ Log important actions and errors



SENIOR ENGINEER TIP
Automation is part of your production environment.

A small script can delete data, expose secrets, or take down services. Treat every script and automation like production code with the same level of review and testing.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Dangerous Patterns Senior Reviewers Catch

0.0.0.0/0, wildcard permissions, latest tags, unpinned dependencies, disabled TLS verification, destructive Terraform changes, missing limits, privileged containers, silent failures, hard-coded credentials, unsafe shell expansion, and hidden single points of failure.

1. 0.0.0.0/0 (OPEN ACCESS)



```
cidr_blocks = [
  "0.0.0.0/0"
]
```

Allows access from anywhere on the internet.

2. WILDCARD PERMISSIONS



Effect: "Allow"
Action: "*"
Resource: "*"

Grants excessive permissions and increases blast radius.

3. LATEST TAGS



```
image: nginx:latest
```

Can pull unexpected versions and break deployments.

4. UNPINNED DEPENDENCIES



```
pip install requests
# No version specified
```

May introduce breaking changes or vulnerable versions.

5. DISABLED TLS VERIFICATION



```
requests.get(
  "https://api",
  verify=False
)
```

Makes connections vulnerable to man-in-the-middle attacks.

6. DESTRUCTIVE TERRAFORM CHANGES



```
- resource "aws_s3_bucket"
  bucket = "prod-data"
```

Can delete or replace critical infrastructure.

7. MISSING LIMITS



```
resources:
  # no limits set
```

Can lead to resource exhaustion, throttling, and node instability.

8. PRIVILEGED CONTAINERS



```
securityContext:
  privileged: true
```

Gives excessive access to the host and increases risk.

9. SILENT FAILURES



```
command || true
```

Hides failures and allows broken deployments to succeed.

10. HARD-CODED CREDENTIALS



```
password = "MyPass123"
api_key = "abc123"
```

Exposes sensitive credentials in code and version control.

11. UNSAFE SHELL EXPANSION



```
rm -rf $DIR
cp $FILE $DEST
```

Can cause unintended file deletion or command injection.

12. HIDDEN SINGLE POINTS OF FAILURE



No redundancy, single database, single AZ, or unmanaged dependencies can cause outages.

13. REAL EXAMPLE: MULTIPLE ISSUES IN ONE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  template:
    spec:
      containers:
        - name: app
          image: myapp:latest
          securityContext:
            privileged: true
          env:
            - name: API_KEY
              value: "abc123"
          command: ["/bin/sh", "-c", "./deploy.sh || true"]
```

Using latest tag

Privileged container

Hard-coded credential

Silent failure (|| true)

14. IMPROVED EXAMPLE

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  template:
    spec:
      containers:
        - name: app
          image: myapp:1.2.3 # pinned version
          resources:
            requests:
              cpu: "100m"
              memory: "256Mi"
            limits:
              cpu: "500m"
          securityContext:
            runAsNonRoot: true
          envFrom:
            - secretRef:
                name: app-secrets
          command: ["/bin/sh", "-c", "./deploy.sh"]
```

15. REVIEW CHECKLIST

- ✓ No 0.0.0.0/0 or overly broad access
- ✓ No wildcard permissions
- ✓ No latest tags
- ✓ Dependencies are pinned
- ✓ TLS verification is enabled
- ✓ No destructive infrastructure changes
- ✓ Resource limits are set
- ✓ Containers are not privileged
- ✓ Failures are not hidden
- ✓ No hard-coded credentials
- ✓ Shell scripts are safe
- ✓ No hidden single points of failure



SENIOR ENGINEER TIP

These patterns often look "innocent" in code reviews but can cause major incidents in production. Always check for them explicitly.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

How to Review a Production Incident Fix

Avoid approving the first plausible patch. Validate the failure hypothesis, evidence, containment, root cause, regression risk, affected failure domain, observability, and recurrence prevention. Separate mitigation from permanent correction.

1. KEY AREAS TO REVIEW

-  **Failure Hypothesis**
Is the root cause clearly defined and supported by evidence?
-  **Evidence**
Are logs, metrics, traces and timelines provided?
-  **Containment**
Was the incident contained before the fix? Is the system stable now?
-  **Root Cause**
Does the fix address the actual root cause and not just the symptom?
-  **Regression Risk**
Could the fix introduce new issues or break existing functionality?
-  **Affected Failure Domain**
What services, regions, or users were affected?
-  **Observability**
Will we be able to detect if this issue happens again?
-  **Recurrence Prevention**
Are additional changes or controls needed to prevent a repeat?
-  **Mitigation vs Correction**
Is this a temporary mitigation or a permanent fix? Both may be needed.

2. INCIDENT FIX REVIEW FLOW



3. EXAMPLE: INCIDENT FIX REVIEW

BEFORE (INCIDENT)	PROPOSED FIX
 Service: payment-api	 Increase DB connection pool size
 Issue: High error rate (500 errors)	 Add proper connection timeouts and retries
 Cause: Database connection pool exhausted	 Add health checks and monitoring
 Impact: Checkout failures for 45 minutes	 Update runbook
 Status: Contained (traffic reduced)	 Test in staging before full rollout

4. MITIGATION vs PERMANENT FIX

MITIGATION (Temporary)	PERMANENT CORRECTION
 ✓ Stop the immediate impact ✓ May not fully fix the root cause ✓ Useful to restore service quickly ✓ Still need a permanent fix	 ✓ Addresses the root cause ✓ Prevents recurrence ✓ Includes long-term improvements (e.g., code, configuration, monitoring) ✓ Should be tracked until completion

5. KEY QUESTIONS TO ASK

- ? What is the exact failure hypothesis?
- ? What evidence supports this hypothesis?
- ? Was the incident properly contained?
- ? Does the fix address the root cause?
- ? What is the regression risk?
- ? Which services, regions, or users were affected?
- ? How will we detect this issue if it happens again?
- ? Is this a mitigation, a permanent fix, or both?
- ? Have we tested the fix in a safe environment?
- ? What additional actions are needed to prevent recurrence?

6. BEST PRACTICES

- ✓ Do not approve the first plausible patch.
- ✓ Require evidence and clear root cause analysis.
- ✓ Verify the incident is contained and stable.
- ✓ Check for regression risk and blast radius.
- ✓ Ensure observability and alerts are in place.
- ✓ Validate the fix in staging or canary first.
- ✓ Separate mitigation from permanent correction.
- ✓ Track follow-up actions until completion.
- ✓ Update runbooks and documentation.
- ✓ Conduct a post-incident review for learnings.

7. COMMON RED FLAGS

- ! No clear root cause or evidence.
- ! Fix only hides the symptom.
- ! Fix not tested in a safe environment.
- ! High regression risk not addressed.
- ! No monitoring or alerting updates.
- ! Patch introduces new security risks.
- ! Rollback plan is missing.
- ! Temporary fix treated as permanent.
- ! No follow-up actions or documentation.
- ! Same issue has happened before.



SENIOR ENGINEER TIP

A good incident fix does more than restore service. It reduces the chance of recurrence, improves observability, and strengthens the system so the same failure is harder to repeat.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

How Senior Engineers Leave Review Comments

Distinguish blocker, risk, question, recommendation, and nit. Explain consequence and reasoning instead of issuing unexplained commands. Challenge designs without attacking authors. Know when a review belongs in an architecture discussion rather than a PR thread.

1. COMMENT TYPES

 BLOCKER	Must be fixed before merge. High chance of failure, security issue, or major operational risk.
 RISK	Should be addressed. Can cause problems in production or create technical debt.
 QUESTION	Needs clarification or more information. Not a change request yet.
 RECOMMENDATION	Suggested improvement. Not strictly required, but adds value.
 NIT	Minor issue (style, formatting, small improvement). Does not block the PR.

2. EXAMPLE REVIEW COMMENTS

 BLOCKER	Blocking: This opens the database to the internet (0.0.0.0/0). This can allow unauthorized access and lead to data breach. Please restrict to specific CIDR ranges (e.g., office/VPC only).
 RISK	Risk: This uses the latest tag for the Docker image. This can pull unexpected changes and break the deployment. Please pin to a specific version (e.g., nginx:1.25.3).
 QUESTION	Question: How will the application handle a failed payment response from the external API? Do we have retries and a circuit breaker in place?
 RECOMMENDATION	Recommendation: Consider moving this logic to a separate module. It will be easier to test, maintain, and reuse.
 NIT	Nit: Use meaningful variable names (e.g., user_count instead of uc). This improves readability.

3. GOOD vs BAD COMMENTS

BAD COMMENT 	GOOD COMMENT 
"Change this." "Use a different image." "This is wrong." "Fix it." "Don't do this." ✗ No explanation ✗ No context ✗ Feels like a command ✗ Can create friction ✗ Hard to learn from	"This can expose the service to the internet. Please restrict it to the VPC. See our network policy document for details." ✓ Explains the consequence ✓ Provides reasoning ✓ Suggests a better approach ✓ Links to documentation (if applicable) ✓ Helps the author learn ✓ Keeps the conversation respectful

4. CHALLENGE DESIGNS THE RIGHT WAY

DON'T	DO
✗ Attack the author ✗ Use negative or emotional language ✗ Dismiss ideas without discussion ✗ Focus only on what is wrong	✓ Challenge the idea, not the person ✓ Ask questions and understand trade-offs ✓ Provide evidence and alternatives ✓ Keep the discussion respectful and technical
EXAMPLE 	"I'm concerned this design will create a single point of failure. Have we considered using multiple AZs or introducing a load balancer? Let's discuss the trade-offs."

5. WHEN TO MOVE TO AN ARCHITECTURE DISCUSSION

 CONSIDER AN ARCHITECTURE DISCUSSION INSTEAD OF A PR THREAD WHEN:
✓ The change affects system design or architecture ✓ There are multiple valid approaches ✓ It involves significant trade-offs (e.g., cost, latency, reliability) ✓ It requires input from multiple teams ✓ The discussion is getting long and complex

6. SENIOR REVIEWER MINDSET

	✓ Think about production impact ✓ Consider security, reliability, and maintainability ✓ Look at the bigger picture, not just the diff ✓ Help the team build better systems ✓ Share knowledge and context ✓ Be consistent and fair
---	---

7. KEY TAKEAWAYS

	✓ Use clear comment types. ✓ Explain the why, not just the what. ✓ Be respectful and constructive. ✓ Challenge designs with evidence. ✓ Know when to move discussions to an architecture forum. ✓ Good reviews improve code, systems, and teams.
---	--



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

The Senior DevOps Approval Gate

Final production checklist: intent understood, blast radius known, IaC verified, security reviewed, failure modes considered, observability present, deployment safe, rollback credible, tests meaningful, ownership clear, and residual risk explicitly accepted.

1. INTENT UNDERSTOOD



- ✓ Purpose of the change is clear
- ✓ Business and technical goals are understood
- ✓ Scope of the change is defined
- ✓ Success criteria are clear

2. BLAST RADIUS KNOWN



- ✓ Affected services and systems are identified
- ✓ User, region, and environment impact is known
- ✓ Dependencies are mapped
- ✓ Maximum potential impact is documented

3. IaC VERIFIED



- ✓ Infrastructure changes reviewed
- ✓ Plans show expected changes
- ✓ No unintended resource destruction
- ✓ State and module versions are correct

4. SECURITY REVIEWED



- ✓ Access, permissions and roles are correct
- ✓ Secrets are handled securely
- ✓ No hard-coded credentials
- ✓ Compliance and policy requirements are met

5. FAILURE MODES CONSIDERED



- ✓ What can go wrong is identified
- ✓ Dependency failures considered
- ✓ Timeouts, retries and limits reviewed
- ✓ No single point of failure introduced

6. OBSERVABILITY PRESENT



- ✓ Metrics, logs and traces in place
- ✓ Dashboards updated if needed
- ✓ Alerts configured and tested
- ✓ Deployment markers added
- ✓ We can detect issues quickly

7. DEPLOYMENT SAFE



- ✓ Deployment strategy is appropriate (rolling, blue-green, canary, etc.)
- ✓ Change is backward compatible
- ✓ Feature flags used where possible
- ✓ Deployment steps are tested in a non-production environment

8. ROLLBACK CREDIBLE



- ✓ Rollback plan is documented
- ✓ Rollback has been tested
- ✓ No irreversible operations
- ✓ Data rollback steps are defined (if needed)

9. TESTS MEANINGFUL



- ✓ Automated tests cover key changes
- ✓ Integration and end-to-end tests pass
- ✓ Tests reflect real-world scenarios
- ✓ Manual testing completed (if required)

10. OWNERSHIP CLEAR



- ✓ Service owner is identified
- ✓ On-call team is aware
- ✓ Support runbooks are updated
- ✓ Responsibilities during and after the deployment are clear

11. RESIDUAL RISK ACCEPTED



- ✓ Remaining risks are documented
- ✓ Risks are discussed with stakeholders
- ✓ Business impact is accepted
- ✓ Approval is recorded with clear conditions (if any)

12. READY FOR APPROVAL



- ✓ All checklist items completed
- ✓ Open questions resolved
- ✓ Required stakeholders have reviewed
- ✓ We are confident to proceed



SENIOR QUESTION

If this fails in production tonight, do we know how we will detect, contain, diagnose, and recover from it?



SENIOR ENGINEER TIP

Approval is not about trusting that it will work. It is about being prepared when it does not.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta